RTL Design Sherpa

CELL Micro-Architecture Specification 0.12

October 19, 2026

Table of Contents

1 Cell Mas Index..... 9

2 Document Information..... 10

2.1 Overview..... 10

2.1.1 CELL Micro-Architecture Specification..... 10

2.1.2 Document Purpose..... 10

2.1.3 Intended Audience..... 10

2.2 Design Notes..... 11

2.2.1 Revision History..... 11

2.3 References..... 13

2.3.1 Related Documents..... 13

3 Design Philosophy and Scope..... 14

3.1 What This Block Is..... 14

3.2 Scope and Deferrals..... 14

3.3 MAS Decisions at a Glance..... 15

3.4 Relationship to the ALU..... 16

4 Module Tree..... 18

4.1 Purpose..... 18

4.2 Module Tree..... 18

4.3 Block Intent (one paragraph each)..... 18

4.3.1 delta_mesh_cell..... 18

4.3.2 cell_rx..... 18

4.3.3 cell_cfg_regs..... 19

4.3.4 cell_mem_in....19
4.3.5 cell_mem_out....19
4.3.6 cell_credit....19
4.3.7 cell_packetizer....19
4.3.8 cell_exec_fsm....19
4.4 Parameters....20
4.5 Clock and Reset....20
5 Datapath Figures....22
5.1 Whole-Cell Datapath....22
5.1.1 Figure 2.1: Whole-cell datapath....22
5.2 Credit Loop....23
5.2.1 Figure 2.2: Credit loop....23
5.3 ALU Datapath Focus....24
5.3.1 Figure 2.3: ALU datapath focus....24
6 External and Internal Interfaces....26
6.1 Cell Boundary....26
6.1.1 Mesh-Facing Ports....26
6.1.2 Local Port Contract....27
6.1.3 AXI4-Lite Register Interface....27
6.1.4 Observability....27
6.2 Internal Interfaces....28
6.2.1 cell_rx → cell_mem_in....28
6.2.2 cell_rx → cell_cfg_regs....28
6.2.3 cell_cfg_regs → datapath....29

6.2.4 cell_mem_in → cell_exec_fsm....30
6.2.5 cell_exec_fsm → cell_mem_out....30
6.2.6 cell_exec_fsm → cell_credit....30
6.2.7 cell_credit → node (credit VC)....31
6.2.8 cell_mem_out → cell_packetizer....31
6.2.9 cell_packetizer → node (data network)....31
6.2.10 cell_exec_fsm ↔️ ALU....31
7 Cycle Behavior: Fill Path as Built....32
7.1 Slice-1 Fill Path....32
7.1.1 Head-Beat Reception....32
7.1.2 Coordinate Match....32
7.1.3 Demux Decision....32
7.1.4 Payload Write Timing....33
7.1.5 Occupancy Accounting....33
7.2 Expected Slice-2/3 Cycle Behavior....33
8 Credit Logic....35
8.1 MAS Decision: Memory Sizing and Advertisement....35
8.2 MAS Decision: Credit Batch Size and Timeout....35
8.3 Per-Source Owed Counters....35
8.4 Credit Packet Format....36
8.5 Boot Advertisement Packet....36
8.6 Sender-Generic Returns....37
8.7 Implementation Note: Counter Bank Sizing....37
9 Packetizer and Execution FSM....38

9.1 Packetizer....38
9.1.1 MAS Decision: Batching and Flush Conditions....38
9.1.2 MAS Decision: Strap Counter Initialization....38
9.1.3 Header Build....38
9.1.4 Destination and Tag Programming....39
9.1.5 Strap Accounting....39
9.2 Execution FSM....40
9.2.1 Pop → Issue → Push Loop....40
9.2.2 MAS Decision: Operand-Source Mux....40
9.2.3 IP/OP Pointer Behavior....40
9.2.4 Gatling as the Cell ALU Sequencer (owner ruling 2026-10-19)....41
9.2.5 ALU Hookup....41
9.2.6 Issue Tag and Same-Tag Alignment (ratified spec — RTL pending)....42
9.2.7 MAC Accumulation....42
9.2.8 Soft Reset....42
10 Verification Plan....43
10.1 House Test Grid....43
10.2 Existing Tests....43
10.3 Planned Slice-2 Credit-Loop Test....43
10.4 Planned Slice-3 ALU-Run Test....44
10.5 Verification Hooks....44
10.6 Assertions and Checks....45
11 Register Map and MAS Decisions....46
11.1 Register Map....46

11.2 Bit Fields....47
11.2.1 CTRL (0x00)....47
11.2.2 STATUS (0x04) — read-only....48
11.2.3 FUNC_SEL (0x08)....48
11.2.4 OP_SRC (0x34)....48
11.2.5 IRQ_STATUS / IRQ_EN....49
11.3 MAS Decisions Revisited....49
11.4 RTL Naming Divergence (slice 1)....50
12 References....51
12.1 Normative Specifications....51
12.2 RTL Sources....51
12.3 Verification....52
12.4 House Style and Build....52

List of Figures

Figure 2.1: Whole-cell datapath....22
Figure 2.2: Credit loop....23
Figure 2.3: ALU datapath focus....24

List of Tables

Table 1: Document information....10
Table 2: Revision history....11
Table 3: Table 1.1: Summary of MAS decisions....15
Table 4: Table 2.1: cell_top parameters....20
Table 5: Table 3.1: Mesh-facing AXI4-Stream ports....26
Table 6: Table 3.2: Observability outputs....27
Table 7: Table 3.3: Input-memory write port....28
Table 8: Table 3.4: Config-stream register interface....28
Table 9: Table 3.5: Register outputs to datapath....29
Table 10: Table 3.6: Input-memory read port....30
Table 11: Table 3.7: Result-memory write port....30
Table 12: Table 3.8: Consumed-line credit strobe....30
Table 13: Table 3.9: Result-memory read port....31
Table 14: Table 4.1: Slice-1 head-beat dispatch....32
Table 15: Table 4.2: Result-packet header fields....38
Table 16: Table 4.3: OP_SRC encoding....40
Table 17: Table 4.4: ALU↔FSM seam signals....41
Table 18: Table 5.1: House test grid....43
Table 19: Table 5.2: Verification hooks....44
Table 20: Table 6.1: Cell register map....46
Table 21: Table 6.2: CTRL bit fields....47
Table 22: Table 6.3: STATUS bit fields....48
Table 23: Table 6.4: FUNC_SEL fields....48
Table 24: Table 6.5: OP_SRC fields....48
Table 25: Table 6.6: IRQ fields....49
Table 26: Table 6.7: MAS decisions and their hardware homes....49

1 Cell Mas Index

Generated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 Overview

2.1.1 CELL Micro-Architecture Specification

Document information

Property	Value
Document Title	CELL Micro-Architecture Specification
Version	0.12
Date	2026-10-19
Status	Draft — slice-1 RTL built; slice-2/3 modules specified
Classification	Open Source - MIT License

2.1.2 Document Purpose

This Micro-Architecture Specification (MAS) documents how the compute cell is built and how the remaining slice-2/3 work completes it:

- The slice-1 module tree (`cell_top`, `cell_rx`, `cell_cfg_regs`, `cell_mem_in`, `cell_mem_out`) as it exists today
- The slice-2/3 modules (`cell_credit`, `cell_packetizer`, `cell_exec_fsm`) that are specified here
- The MAS decisions that close the PRD §12 open questions
- The cycle-level behavior of the fill path as built and the expected consume/emit path
- The verification plan for the credit loop and ALU-run tests

For what the cell must do and why, see the **cell PRD** (normative for the cell product). For the packet, credit, and coordinate protocols the cell consumes, see **TILE_SPEC §11**. For the ALU block itself, see the **alu-infer PRD** and **alu-infer MAS**.

2.1.3 Intended Audience

- RTL designers implementing the slice-2/3 modules inside `cell/rtl/`
- Verification engineers extending `cell/dv/tests/test_cell.py`
- Design reviewers checking the future implementation against intent
- Students reading the cell datapath top to bottom

2.2 Design Notes

2.2.1 Revision History

Revision history

Version	Date	Author	Description
0.11	2026-10-18	RTL Design Sherpa	Beat-chained header rule (owner ruling): the 128-bit tile header occupies the first `ceil(128/DW)` payload beats, LSB-first; every link keeps its own width. Local port stays at delta `DATA_WIDTH` (64b) — supersedes 0.10's 128-bit local-port widening. TILE_SPEC v1.6+ binding.
0.12	2026-10-19	RTL Design Sherpa	(addendum) Owner rulings 2026-10-18/19: local-port width stays at delta DATA_WIDTH (64b) with beat-chained header; strap terminology replaces grants in registers/PRD; DEST reserved, FLUSH_DEST/FLU

Version	Date	Author	Description
			SH_TAG are the operational drain destination; credit-class encoding added; gatling sequences the cell ALU. Slice-1 RTL identifiers retain pre-ruling grant names (noted in ch06).
0.11	2026-10-18	RTL Design Sherpa	(same-day addendum) ALU-focus datapath figure (fig 2.3, no delta side) + issue-tag / same-tag-alignment MAS decision in ch04 §4.4 — ratified owner ruling; tag assign/FIFO/align gate not yet in alu-infer RTL.
0.10	2026-10-18	RTL Design Sherpa	First issue. Slice-1 RTL built; slice-2/3 specified. Closes PRD §12 open questions with MAS decisions: IN_DEPTH=OUT_DEPTH=32 lines; CREDIT_BATCH_N default 1; 64-cycle partial-batch credit timeout;

Version	Date	Author	Description
			EMIT_BATCH_X flush on batch-full or CTRL.flush-and-send; strap counter default 16; local-port data width widens to 128 bits to carry the §11.6 tile header.

2.3 References

2.3.1 Related Documents

- **cell PRD** (../../../PRD.md) — product definition, normative for the cell
- **TILE_SPEC §11** (../../../../tile/docs/TILE_SPEC.md) — packet header, transport, credits/grants, coordinates
- **alu-infer PRD** (../../../../alu-infer/PRD.md) — normative definition of the ALU block
- **alu-infer MAS** (../../../../alu-infer/docs/alu_infer_mas/alu_infer_mas_index.md) — implementation-intent book for the ALU block
- **delta_mesh_cell** (../../../../../fabric-gen-ip/delta/rtl/delta_mesh_cell.sv) — the node the cell instantiates

---

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Design Philosophy and Scope

3.1 What This Block Is

The **cell** is the tile's compute unit: sixteen of them make a tile. Each cell contains exactly one `delta_mesh_cell` node, instantiated inside `cell_top`, with its local port (port index 8, `P_LOCAL`) facing the cell's internal datapath and its eight mesh-facing ports passing through to the cell boundary. That containment is the entire cell↔delta relationship: the node is governed by delta's own HAS/MAS, and everything else in the cell is the cell's own design, speaking the tile-level contracts of `TILE_SPEC` §11.

The cell's job is to turn delta-delivered operand lines into result lines that leave back through the delta mesh toward RAPIDS:

1. **Program.** `cfg` packets write the §7 registers — `FUNC_SEL`, `OP_SRC`, batching knobs, and the persistent operands `A`–`D`, `T`.
2. **Fill.** `mem` packets write operand lines into the input memory, each line tagged with the packet's source coordinate.
3. **Consume.** The execution FSM pops `OP_LINES` input lines per invocation, issues them to the ALU, and pushes result lines into the result memory.
4. **Credit return.** As each input line is consumed, the credit logic returns credits to the recorded source.
5. **Emit.** The packetizer batches result flits into packets and sends them toward the gatling-programmed drain destination, spending the cell's `TILE_LAND_FLITS` strap counter.

Everything — data, config, and programming — arrives in-band on delta (PRD §3; TILE_SPEC §11.13; alu-infer PRD §5).

3.2 Scope and Deferrals

In scope for this MAS:

- The slice-1 RTL as built: `cell_top`, `cell_rx`, `cell_cfg_regs`, `cell_mem_in`, `cell_mem_out`
- The slice-2/3 modules specified here: `cell_credit`, `cell_packetizer`, `cell_exec_fsm`
- The ALU↔FSM seam (the ALU itself is specified in the alu-infer MAS)
- The credit-loop and result-packetization behavior
- The verification plan for the credit-loop and ALU-run tests

Out of scope or deferred (PRD §10–11):

- Tetrad/root node internals and the tetrad control core
- RAPIDS internals and the CXS migration sequencing
- Fabric-level Z stacking
- Cell-destined result sending (`DEST` override) pending the tile-spec cell→cell grant extension
- Loading `A`–`D` from the input memory
- Wider-than-32-bit lines and element widths other than bf16

3.3 MAS Decisions at a Glance

This book closes the PRD §12 open questions with the following implementation decisions, each marked “MAS decision” with rationale in the chapter where it lives:

Table 1.1: Summary of MAS decisions

Decision	Value	Rationale
Input memory depth (`IN_DEPTH`)	32 lines	Matches result memory depth; gives a 32-flit advertisement today; simple power-of-two addressing
Result memory depth (`OUT_DEPTH`)	32 lines	Sized to absorb one full input run’s results at the default `OP_LINES`=1; deeper than the default `TILE_LAND_FLITS` strap so the memory is not the emit bottleneck
Line width (`LINE_W`)	32 bits (2×bf16)	bf16 element model per alu-infer PRD §2; slice-3 may pack two lines per 64-bit flit
Boot advertisement	32 flits	One 32-bit line per 64-bit payload flit today; the value is `IN_DEPTH` expressed in flits
Credit batch size (`CREDIT_BATCH_N`)	Default 1	Exact per-line behavior; configurable for lower

Decision	Value	Rationale
		fabric traffic
Partial-batch credit timeout	64 cycles	Bounded drain latency without adding credit traffic at N=1
Result batch size (`EMIT_BATCH_X`)	Configurable, default 4	Amortizes header overhead while respecting the 16-flit strap
Packetizer flush conditions	Batch-full OR `CTRL.flush-and-send`	Simple, deterministic; idle-timeout and end-of-job flush are deferred
Strap counter default	16 flits	Mirrors the Hive default `TILE_LAND_FLITS`; restored by RAPIDS `credit/add` returns
Local-port data width (slice 2/3)	Delta `DATA_WIDTH` (64b today)	Beat-chained header rule (owner ruling 2026-10-18): the 128-bit tile header spans the first `ceil(128/DW)` payload beats, LSB-first; no widening, no width conversion. Supersedes the 0.10 draft's 128-bit local port

3.4 Relationship to the ALU

The cell owns the memories, the FSM, the packetizer, and the credit logic. The ALU block — `alu-infer` — is the cell's compute engine and is governed by its own PRD and MAS. The cell MAS cites the alu-infer contracts at the FSM seam and does not restate the ALU internals.

Last Updated: 2026-10-18

4 Module Tree

4.1 Purpose

`cell_top` is the compute cell top-level. It instantiates one `delta_mesh_cell` node and the cell's own datapath blocks. The module tree below shows the slice-1 blocks that are built and the slice-2/3 blocks that this MAS specifies.

4.2 Module Tree

```
cell_top (built, slice 1)
├── delta_mesh_cell (black box — fabric-gen-ip/delta)
├── cell_rx (built, slice 1) — header parse / demux from the node's
local port
├── cell_cfg_regs (built, slice 1) — AXIL slave + in-band cfg-stream
register file
├── cell_mem_in (built, slice 1) — input operand memory with per-line
source tag
├── cell_mem_out (built, slice 1) — result memory
├── cell_credit (specified, slice 2) — per-source owed counters +
credit packet builder
├── cell_packetizer (specified, slice 2) — result-flit batching +
§11.5 header builder
└── cell_exec_fsm (specified, slice 3) — pop → issue → push loop + ALU
hookup
```

The ALU block itself (`alu-infer`) is instantiated inside `cell_exec_fsm` (or as a sibling driven by it); its specification is owned by the alu-infer PRD and MAS.

4.3 Block Intent (one paragraph each)

4.3.1 `delta_mesh_cell`

The network attachment. The cell instantiates one node; its local port (`P_LOCAL = 8`) delivers traffic to `cell_rx`, and its eight mesh ports pass through to the cell boundary. The node is a black box owned by `fabric-gen-ip/delta`; its HAS/MAS govern it. The cell depends only on the node's local-port AXI4-Stream contract and on the node's ability to route packets by destination coordinate.

4.3.2 `cell_rx`

Header parse and demux for the node's local port. Slice 1 already parses a provisional 64-bit TUSER packing (because the instantiated delta node exposes a 64-bit TUSER) and routes `mem` payloads to the input memory, `cfg` writes to the register file, and `credit` packets to a counted TODO drop. Slice 2/3 reworks `cell_rx` to the §11.6 transport: delta's route header rides TUSER,

and the 128-bit tile header is the first data beat. Every input-memory line is tagged with the packet's source coordinate so credit returns can be addressed correctly.

4.3.3 cell_cfg_regs

AXI4-Lite slave plus in-band `cfg`-stream register file per PRD §7. The register map is defined in Chapter 6. The block exposes register values to the execution FSM and packetizer, generates the one-cycle `soft_rst` pulse, and assembles `STATUS` and `IRQ_STATUS`.

4.3.4 cell_mem_in

Input operand memory. Sized to `IN_DEPTH` lines; each line carries a source-coordinate tag. It has one write port (from `cell_rx`) and one read port (from `cell_exec_fsm`). The read pointer `IP` is owned by the FSM; occupancy is exposed for `STATUS`.

4.3.5 cell_mem_out

Result memory. Sized to `OUT_DEPTH` lines; one write port (from `cell_exec_fsm`) and one read port (from `cell_packetizer`). The write pointer `OP` is owned by the FSM; occupancy is exposed for `STATUS`.

4.3.6 cell_credit

Credit-return logic. Maintains per-source owed counters keyed by the source coordinate recorded on each consumed input line. Batches returns to `N = CREDIT_BATCH_N` consumed lines per source; drains a partial batch after a 64-cycle timeout. Builds header-only `credit`/`add` packets addressed to the recorded source and sends them back through the node on the credit VC. Sends the boot `credit`/`set` advertisement to `level` 3 carrying the input memory's size in flits.

4.3.7 cell_packetizer

Result drain and packetizer. Batches result flits up to `EMIT_BATCH_X`; flushes on batch-full or on `CTRL.flush-and-send`. Builds the §11.5 tile header with destination and tag supplied by the gatling-programmed drain command, and sends the packet through the node only while the `TILE_LAND_FLITS` strap counter covers the whole packet. Spends the strap counter per flit sent (header included) and restores it as RAPIDS returns `credit`/`add` packets.

4.3.8 cell_exec_fsm

Execution FSM and ALU hookup. Runs the pop→issue→push loop: pops `OP_LINES` input lines per invocation (advancing `IP`), presents them to the ALU's valid/ready elastic pipeline, and pushes each retired result line into the result memory (advancing `OP`). Selects operand sources per `OP_SRC` (input-memory pop or one of `A`/`B`/`C`/`D`; `T` backs the threshold immediate). Clears `IP`/`OP` on `CTRL.soft_reset`.

4.4 Parameters

Table 2.1: `cell_top` parameters

Parameter	Default	Meaning
`LINE_W`	32	Line width in bits; one line is two bf16 lanes
`COORD_W`	4	Coordinate sub-field width; coordinate is `{ext, level, X, Y, Z}`
`POS_X/Y/Z`	0	This cell's position in the tile grid
`IN_DEPTH`	32	Input memory depth in lines
`OUT_DEPTH`	32	Result memory depth in lines
`AXIL_ADDR_WIDTH`	32	AXIL address width
`AXIL_DATA_WIDTH`	32	AXIL data width
`DATA_WIDTH`	64	Cell↔delta local-port data width. Stays at the delta node's `DATA_WIDTH` (64 today); the 128-bit §11.6 tile header is beat-chained LSB-first across `ceil(128/DATA_WIDTH)` beats. No local-port widening.
`USER_WIDTH`	64	Delta route header width; unchanged until delta widens it
`NUM_VC`	2	Number of delta virtual channels

4.5 Clock and Reset

Single clock domain, active-low async `aresetn` (house `ALWAYS_FF_RST`), matching the tile's single-clock-domain decision (TILE_SPEC §9). Reset clears the memories, registers, pointers, and credit/strap state; `CTRL.soft_reset` clears only `IP`, `OP`, `IRQ_STATUS`, and selected datapath state.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

5 Datapath Figures

5.1 Whole-Cell Datapath

5.1.1 Figure 2.1: Whole-cell datapath

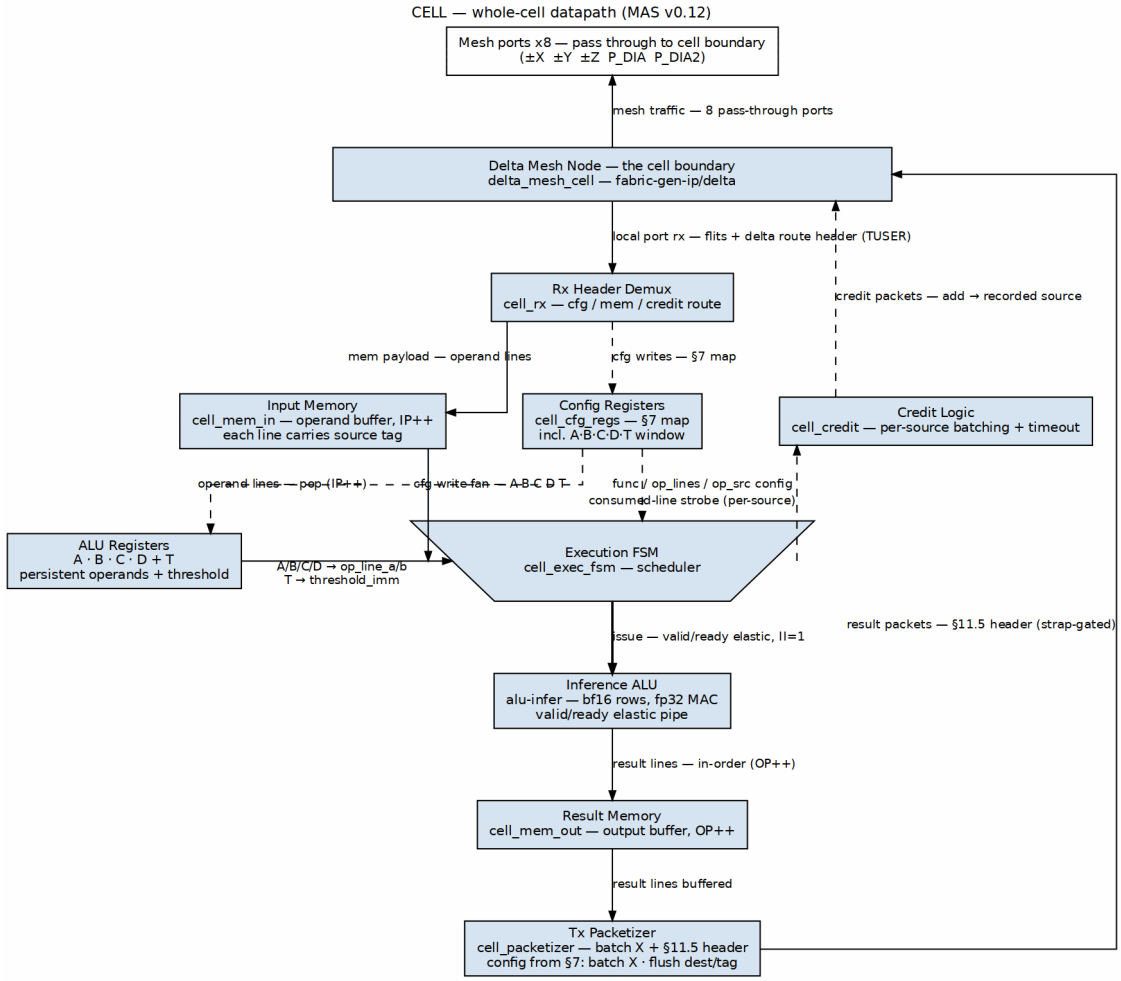


Figure 2.1: Whole-cell datapath

The figure follows the house-style rule that three flows must be traceable at a glance:

- **Programming/config flows down the center:** `cfg` packets enter through the delta node, are demuxed by `cell_rx`, and write the config registers; the registers in turn configure the FSM, the ALU, and the packetizer.
- **Operand fill flows down the left:** `mem` payloads are written into the input memory and are later popped by the execution FSM.

- **Results and credit returns flow back up the right:** result lines move from the ALU into the result memory, through the packetizer, and back out through the delta node; consumed-line credits return on their own dashed path up to the node.

The boundary bar is the delta node edge. Every arrow is labeled with the transaction class it carries; sideband/config arrows are dashed.

5.2 Credit Loop

5.2.1 Figure 2.2: Credit loop

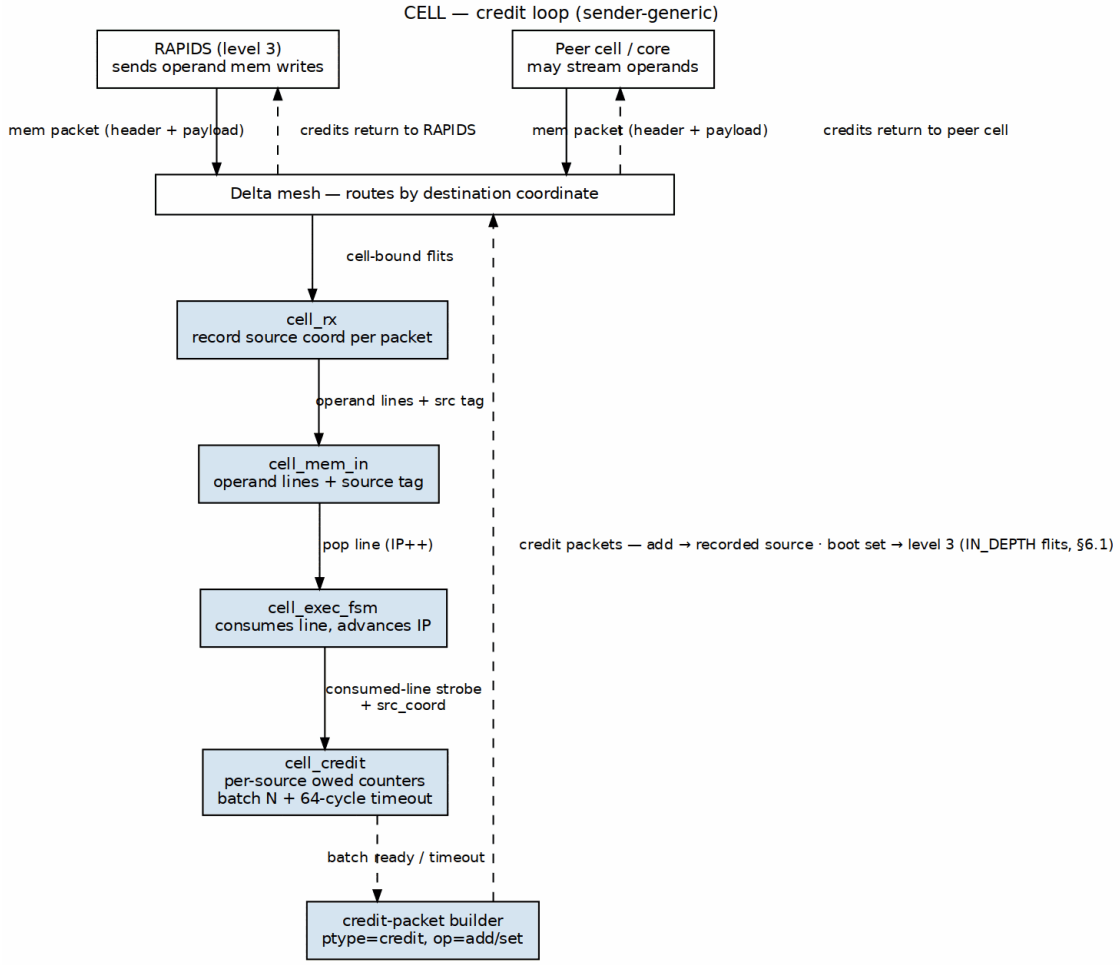


Figure 2.2: Credit loop

The credit-loop figure shows the sender-generic return path:

1. An input line arrives from a source and is tagged with that source's coordinate.

2. The execution FSM consumes the line and strobes `cell_credit`.
3. `cell_credit` increments a per-source owed counter and, when the batch threshold `CREDIT_BATCH_N` is reached (or the 64-cycle timeout expires), sends a header-only `credit`/`add` packet addressed to the recorded source.
4. The packet travels back through the delta node to the source, which restores its per-destination credit counter.

The path never shares an arrow with result data; credits are a separate sideband VC.

5.3 ALU Datapath Focus

5.3.1 Figure 2.3: ALU datapath focus

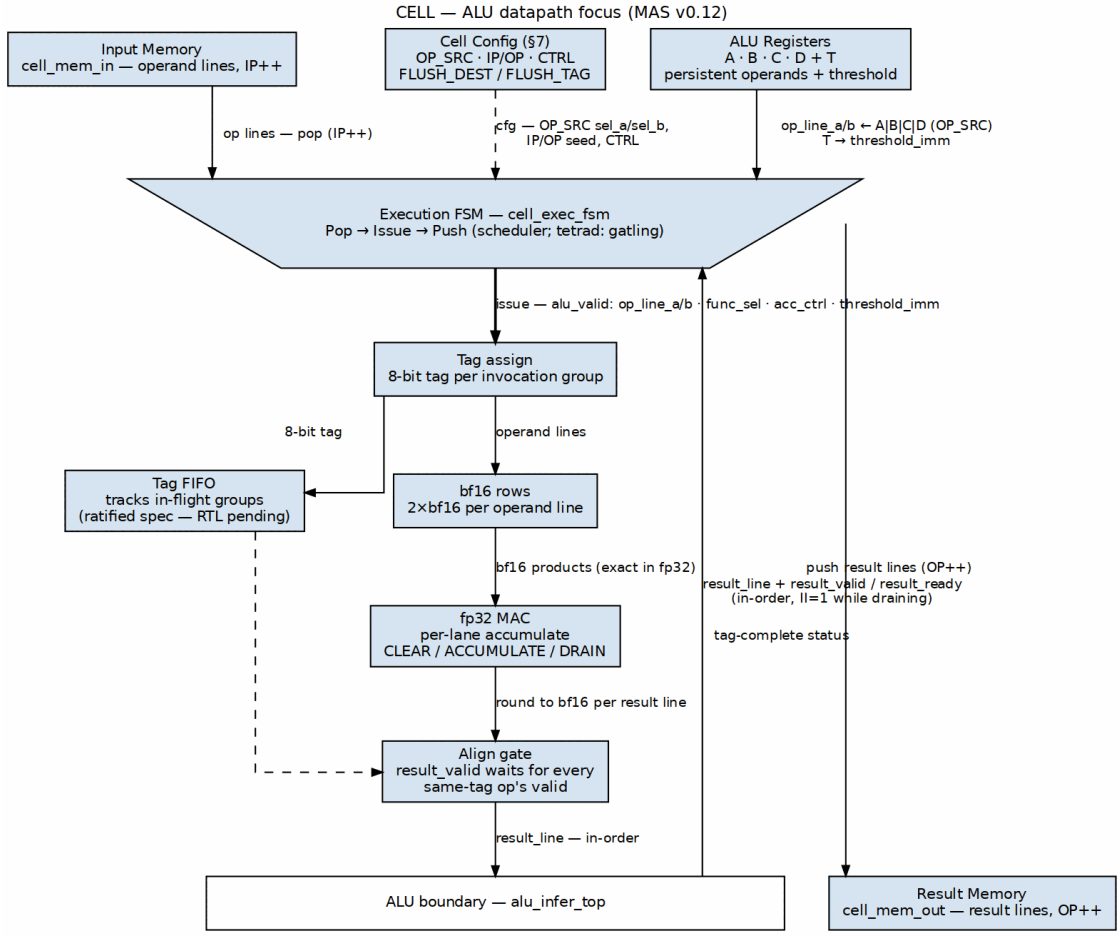


Figure 2.3: ALU datapath focus

The ALU-focus figure drops the delta/network side and traces the three cell-internal flows around `alu_infer_top`:

1. **Operand fill (left):** `cell_mem_in` pops (`IP++`) and the persistent `A·B·C·D/T` registers feed the execution FSM, which owns the `OP_SRC` operand mux. Config (`OP_SRC`, `IP/OP`, `CTRL`) arrives dashed from §7.
2. **Issue (center, heavy pen):** the FSM issues the bundle (`op_line_a/b`, `func_sel`, `acc_ctrl`, `threshold_imm`) with `alu_valid`; the tag assign gives the invocation group its 8-bit tag and the tag FIFO tracks in-flight groups (ratified spec — RTL pending). Rows multiply in bf16 exactly, the fp32 MAC accumulates per-lane under `acc_ctrl`, and the align gate holds `result_valid` until every same-tag op's valid has asserted.
3. **Result flow (right):** `result_line` returns **up** its own route through the ALU boundary bar (`result_valid`/`result_ready`, in-order, II=1 while draining), and the FSM pushes the accepted lines down to `cell_mem_out` (`OP++`).

The single boundary bar is the `alu_infer_top` edge; no flow shares an arrow with another.

Last Updated: 2026-10-18

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 External and Internal Interfaces

6.1 Cell Boundary

6.1.1 Mesh-Facing Ports

`cell_top` exposes the eight mesh-facing ports of the instantiated `delta_mesh_cell` node:

Table 3.1: Mesh-facing AXI4-Stream ports

Port	Direction	Width	Purpose
`s_axis_tdata[8]`	in	`DATA_WIDTH`	Data from mesh neighbors
`s_axis_tvalid[8]`	in	1	Valid from mesh neighbors
`s_axis_tready[8]`	out	1	Ready to mesh neighbors
`s_axis_tlast[8]`	in	1	Packet last from mesh neighbors
`s_axis_tuser[8]`	in	`USER_WIDTH`	Delta route header from mesh neighbors
`m_axis_tdata[8]`	out	`DATA_WIDTH`	Data to mesh neighbors
`m_axis_tvalid[8]`	out	1	Valid to mesh neighbors
`m_axis_tready[8]`	in	1	Ready from mesh neighbors
`m_axis_tlast[8]`	out	1	Packet last to mesh neighbors
`m_axis_tuser[8]`	out	`USER_WIDTH`	Delta route header to mesh neighbors
`s_credit[8]`	in	`NUM_VC`	Per-VC credit levels from neighbors
`m_credit[8]`	out	`NUM_VC`	Per-VC credit

Port	Direction	Width	Purpose
			levels to neighbors

These ports are pass-through wiring in `cell_top`; the node performs all routing. Cell-bound traffic must be addressed to the cell's external ID `{ext=1, level=0, X, Y, Z}` and ride delta QoS ≠0 so the node does not consume it as its own config traffic (TILE_SPEC §11.2).

6.1.2 Local Port Contract

The cell's internal datapath sees only the node's local port (`P_LOCAL = 8`). Slice 1 used a provisional 64-bit TUSER packing for the tile header because the instantiated delta node exposes 64-bit `TUSER`.

MAS decision: Slice 2/3 moves to the `TILE_SPEC` §11.6 transport binding, with the beat-chained header rule (owner ruling 2026-10-18): every link keeps its own data width, and the 128-bit tile header occupies the first `ceil(128/DW)` payload beats, LSB-first.

- The **delta route header** rides `TUSER` of the head beat, as delta owns it.
- The **128-bit tile header** is payload: on the 64-bit local port it spans beats 0–1 (`tdata[63:0]` = header `[63:0]`, next beat's `tdata[63:0]` = header `[127:64]`), then payload beats follow. On a 128-bit-or-wider link it is beat 0 alone.
- The cell's local-port data width **stays at the delta node's `DATA_WIDTH` (64 bits today)** — no widening, no width conversion anywhere. The demux reassembles the two header beats; the packetizer emits the header across two beats.

This supersedes slice-1's provisional 64-bit TUSER packing and the earlier MAS 0.10 draft's 128-bit local-port widening.

6.1.3 AXI4-Lite Register Interface

The cell exposes a house-standard AXI4-Lite slave for debug and local control. All registers are also writable from the in-band `cfg` stream. The register map is in Chapter 6.

6.1.4 Observability

Table 3.2: Observability outputs

Signal	Width	Purpose
`o_err_drop_cnt`	32	Dropped-head count from the delta node
`o_todo_credit_drop_cnt`	32	Slice-1 TODO counter for

Signal	Width	Purpose
		unhandled credit packets
o_todo_cfg_read_drop_cnt	32	Slice-1 TODO counter for unhandled cfg read replies
o_irq	1	Interrupt output (level)
MonBus (AXI4-Lite)	-	The cell's one MonBus stream, merged at the tetrad with the other three cells and the tetrad node (tile spec §5). Not yet in the RTL

6.2 Internal Interfaces

6.2.1 cell_rx → cell_mem_in

Table 3.3: Input-memory write port

Signal	Width	Direction	Purpose
mem_wr_valid	1	rx → mem	Write valid
mem_wr_ready	1	mem → rx	Write ready
mem_wr_addr	log2(IN_DEPTH)	rx → mem	Line index
mem_wr_data	LINE_W	rx → mem	Line data
mem_wr_src_coord	COORD_BITS	rx → mem	Source coordinate tag

6.2.2 cell_rx → cell_cfg_regs

Table 3.4: Config-stream register interface

Signal	Width	Direction	Purpose
cfg_wr_valid	1	rx → regs	In-band cfg write valid
cfg_wr_ready	1	regs → rx	In-band cfg write ready
cfg_wr_addr	8	rx → regs	Register offset
cfg_wr_data	32	rx → regs	Register value

Signal	Width	Direction	Purpose
cfg_rd_valid	1	rx → regs	In-band cfg read request valid (slice-2 completion)
cfg_rd_addr	8	rx → regs	Register offset
cfg_rd_data	32	regs → rx	Read data (slice-2 completion)
cfg_rd_ready	1	rx → regs	Read request accepted (slice-2 completion)

6.2.3 cell_cfg_regs → datapath

The register block drives the following register values:

Table 3.5: Register outputs to datapath

Signal	Width	Purpose
r_ctrl	32	CTRL register
r_func_sel	32	FUNC_SEL register
r_op_lines	32	OP_LINES register
r_emit_batch_x	32	EMIT_BATCH_X register
r_credit_batch_n	32	CREDIT_BATCH_N register
r_irq_en / r_irq_status	32	IRQ enables and status
r_a / r_b / r_c / r_d / r_t	32	Persistent ALU operands and threshold
r_op_src	32	Operand-source mux selection
r_dest	32	Deferred cell-destined destination
r_ip / r_op	32	Input/result memory pointers
o_soft_rst	1	One-cycle soft-reset pulse

6.2.4 `cell_mem_in` → `cell_exec_fsm`

Table 3.6: Input-memory read port

Signal	Width	Direction	Purpose
`mem_in_rd_valid`	1	fsm → mem	Read request valid
`mem_in_rd_ready`	1	mem → fsm	Read request ready
`mem_in_rd_addr`	`log2(IN_DEPTH)`	fsm → mem	Read address
`mem_in_rd_data`	`LINE_W`	mem → fsm	Read data (registered)
`mem_in_rd_src_coord`	`COORD_BITS`	mem → fsm	Source tag of read line
`mem_in_empty`	1	mem → fsm/reg	Memory empty
`mem_in_full`	1	mem → fsm/reg	Memory full
`mem_in_occupancy`	`log2(IN_DEPTH+1)`	mem → fsm/reg	Line occupancy

6.2.5 `cell_exec_fsm` → `cell_mem_out`

Table 3.7: Result-memory write port

Signal	Width	Direction	Purpose
`mem_out_wr_valid`	1	fsm → mem	Write valid
`mem_out_wr_ready`	1	mem → fsm	Write ready
`mem_out_wr_addr`	`log2(OUT_DEPTH)`	fsm → mem	Write address
`mem_out_wr_data`	`LINE_W`	fsm → mem	Result line

6.2.6 `cell_exec_fsm` → `cell_credit`

Table 3.8: Consumed-line credit strobe

Signal	Width	Direction	Purpose
`credit_strobe`	1	fsm → credit	One input line consumed
`credit_src_coord`	`COORD_BITS`	fsm → credit	Source coordinate of consumed line

6.2.7 `cell_credit` → node (credit VC)

`cell_credit` builds header-only `credit/add` and `credit/set` packets and injects them into the node's local-port credit sideband. The packet format is defined in TILE_SPEC §11.5 and §11.9.

6.2.8 `cell_mem_out` → `cell_packetizer`

Table 3.9: Result-memory read port

Signal	Width	Direction	Purpose
`pkt_rd_valid`	1	pkt → mem	Read request valid
`pkt_rd_ready`	1	mem → pkt	Read request ready
`pkt_rd_addr`	`log2(OUT_DEPTH)`	pkt → mem	Read address
`pkt_rd_data`	`LINE_W`	mem → pkt	Result line
`mem_out_empty`	1	mem → pkt	Memory empty
`mem_out_occupancy`	`log2(OUT_DEPTH+1)`	mem → pkt	Line occupancy

6.2.9 `cell_packetizer` → node (data network)

`cell_packetizer` builds write-format packets (`ptype=mem`, `op=0`) with the §11.5 tile header and sends them through the node's local port. The destination and tag are supplied by the gatling-programmed drain command; the source coordinate is the cell's external ID.

6.2.10 `cell_exec_fsm` ↔️ ALU

The ALU interface is defined in the alu-infer MAS; the cell MAS cites it in Chapter 4.3.

Last Updated: 2026-10-18

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 Cycle Behavior: Fill Path as Built

7.1 Slice-1 Fill Path

The slice-1 RTL already implements the receive and store side of the cell. This section documents its cycle behavior so that the slice-2/3 work can be checked against it.

7.1.1 Head-Beat Reception

A packet arrives on the node's local port as a sequence of AXI4-Stream beats. On the head beat:

- `s_user[11:0]` carries the destination `{X, Y, Z}`.
- `s_user[13:12]` carries the QoS class; it must be `2'b01` (non-zero) so the node does not consume the packet as its own config traffic.
- `s_user[29:15]` carries the source coordinate `{ext, level, X, Y, Z}`.
- `s_user[32:30]` carries `ptype`: `cfg=0`, `mem=2`, `credit=3`, ...
- `s_user[33]` carries `op` (write/read or credit add/set).
- `s_user[49:34]` carries a compacted 16-bit length.
- `s_user[59:50]` carries a compacted 10-bit target address.
- `s_user[63:60]` carries the 4-bit tag.

This 64-bit TUSER packing is provisional and exists only because the instantiated delta node exposes a 64-bit TUSER. It is superseded in slice 2/3 by the `TILE_SPEC` §11.6 two-header model.

7.1.2 Coordinate Match

`cell_rx` compares the destination `{X, Y, Z}` against its own `POS_X/Y/Z`. The `ext` bit and QoS are not part of the match; QoS is already required to be non-zero so the node did not consume the packet.

7.1.3 Demux Decision

Table 4.1: Slice-1 head-beat dispatch

ptype	op	Action
`mem`	0 (write)	Accept payload beats into `cell_mem_in` starting at `target_addr`; increment `mem_wr_ptr` per beat
`cfg`	0 (write)	Drive `cfg_wr_valid` to `cell_cfg_regs` with

ptype	op	Action
		target_addr[7:0] and s_data[31:0]
cfg	1 (read)	Counted TODO drop in slice 1; slice 2 builds a reply packet
credit	any	Counted TODO drop in slice 1; slice 2 routes to cell_credit
other	any	Drop head; increment o_err_drop_cnt via inc_err_drop

7.1.4 Payload Write Timing

For a mem write packet:

- Cycle 0: head beat accepted; in_pkt latches pkt_src_coord, pkt_ptype, pkt_target; mem_wr_ptr clears.
- Cycle N: each payload beat with s_valid && s_ready && mem_wr_ready writes mem_data[addr] and mem_tag[addr] and increments mem_wr_ptr, where addr = pkt_target + mem_wr_ptr.
- The last payload beat (s_last) clears in_pkt.

cell_mem_in exposes wr_ready = !o_full; cell_rx backpressures the local port while a mem payload is in flight and the memory is full.

7.1.5 Occupancy Accounting

cell_mem_in increments its occupancy counter on every accepted write and decrements it on every accepted read. The read side is held idle in slice 1 (mem_in_rd_valid = 1'b0 in cell_top), so occupancy only rises during fill tests.

7.2 Expected Slice-2/3 Cycle Behavior

Slice 2/3 keeps the same fill timing but changes the header transport:

- The head beat's TUSER carries only the delta route header.
- The 128-bit tile header is **payload**, beat-chained per the owner ruling 2026-10-18: on the 64-bit local port, beat 0 carries tdata = header[63:0] and beat 1 carries tdata = header[127:64]; payload beats follow. The demux reassembles the header across the two beats before dispatching on ptype.

- Payload beats carry one 32-bit line per 64-bit flit today; slice 3 packs two lines per 64-bit flit.

The demux still records the source coordinate from the tile header's source field (not delta's hop-source route header) and tags each input-memory line with it.

Last Updated: 2026-10-18

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 Credit Logic

8.1 MAS Decision: Memory Sizing and Advertisement

MAS decision: `IN_DEPTH = 32` lines and `OUT_DEPTH = 32` lines.

Rationale: A 32-line depth is a power of two, matches the default credit-strap scale, and gives simple 5-bit pointers. The result memory is intentionally deeper than the default `TILE_LAND_FLITS` strap (16 flits) so that the landing-space strap, not the result memory, is the emit bottleneck. The input memory's flit capacity is the cell's boot advertisement value.

With `LINE_W = 32` and a 64-bit payload flit today, one input line occupies one payload flit (low 32 bits). Therefore the boot advertisement is:

```
advertisement flits = IN_DEPTH = 32 flits
```

Note: Slice 3 may pack two 32-bit lines per 64-bit payload flit; if that packing is adopted, the advertisement value must be re-derived from the new flit count (`IN_DEPTH / 2` flits) while the line count stays 32. Until then, the advertisement is 32 flits.

8.2 MAS Decision: Credit Batch Size and Timeout

MAS decision: Per-source credit batching uses `N = CREDIT_BATCH_N` consumed lines per source, with a default of `N = 1`. A partial batch that has not reached `N` drains after a 64-cycle timeout.

Rationale: `N = 1` gives the simplest exact-per-line behavior, which is what the slice-2 credit-loop test will verify. Making `N` configurable lets later builds trade return latency for lower credit-packet traffic. The 64-cycle timeout bounds the latency of a trailing partial batch without adding any traffic at `N = 1`.

8.3 Per-Source Owed Counters

`cell_credit` maintains a bank of owed counters indexed by the recorded source coordinate. Each counter has:

- `owed[src]`: number of consumed lines not yet returned to this source
- `timer[src]`: cycles since the last return to this source (valid when `owed` is non-zero and below the batch threshold)

On every consumed-line strobe from `cell_exec_fsm`:

1. `owed[src] += 1`
2. If `owed[src] >= CREDIT_BATCH_N`, send one `credit/add` packet for `CREDIT_BATCH_N` flits, decrement `owed[src]` by `CREDIT_BATCH_N`, and clear the timer.

3. Otherwise, load the timer with the timeout value (or reset it if it was already running).

Every cycle, for each source with `0 < owed[src] < CREDIT_BATCH_N`:

- If `timer[src] == 0`, send one `credit`/`add` packet for `owed[src]` flits and clear `owed[src]`.
- Else, `timer[src] -= 1`.

The timeout value is 64 cycles. The timer is loaded only when a new partial batch begins; additional consumed lines in the same partial batch do not reset the timer (they will be included in the same return when it fires).

8.4 Credit Packet Format

A consumed-line credit return is a header-only `credit` packet with:

- `ptype = credit` (3)
- `op = 0` (add)
- `length` = number of flits returned (`CREDIT_BATCH_N` or the residual `owed` count)
- `destination` = the recorded source coordinate of the consumed line
- `source` = this cell's external ID
- `tag` = echoed or zero (the sink does not store credit packets)
- `target_address[1:0] = credit_class = 0` (sink space)

Credit packets ride the credit VC and need no grant (TILE_SPEC §11.9). TILE_SPEC §11.5 encodes the credit class in the low two bits of the target address: `0` = sink-space credit, `1` = descriptor-slot credit. Consumed-line returns and the boot advertisement below are sink-space credits (class 0).

8.5 Boot Advertisement Packet

After reset, `cell_credit` sends one `credit`/`set` packet to `level = 3` (the tile I/O / RAPIDS) carrying the input memory's full size in flits:

- `op = 1` (set)
- `length = IN_DEPTH` flits (32 today)
- `destination = {ext=0, level=3, X/Y/Z = tile I/O}`
- `source` = this cell's external ID
- `target_address[1:0] = credit_class = 0` (sink space)

This advertisement is the cell's "alive" signal; RAPIDS's external sink-space counter for this cell starts at 0 and is filled by this packet (TILE_SPEC §11.10).

8.6 Sender-Generic Returns

The return is sender-generic: a credit goes to the source coordinate recorded on the input-memory line, not to a hard-coded RAPIDS ID. A cell that streamed into this cell's input memory therefore receives the credits, not RAPIDS, exactly as TILE_SPEC §11.9 requires.

8.7 Implementation Note: Counter Bank Sizing

The number of legal source coordinates in a tile is bounded by the credit-counter index table in TILE_SPEC §11.9. A cell therefore needs at most enough per-source owed counters to cover every possible source that may send to it. A practical implementation allocates one counter per external ID (RAPIDS + peer cells + tetrad cores) that the tile can address, lazily allocating on first strobe if area must be minimized.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 Packetizer and Execution FSM

9.1 Packetizer

9.1.1 MAS Decision: Batching and Flush Conditions

MAS decision: The packetizer batches result flits up to `EMIT_BATCH_X` and flushes on either of two conditions:

1. The batch accumulator holds `EMIT_BATCH_X` flits.
2. `CTRL.flush-and-send` is written (bit 2 of `CTRL`).

An idle-timeout flush and an end-of-job marker flush are documented as deferred (PRD §12; PRD §11). This keeps slice-2 behavior deterministic and testable: a testbench can either fill a batch or force a flush.

9.1.2 MAS Decision: Strap Counter Initialization

MAS decision: The packetizer maintains a strap counter initialized from a cell parameter `TILE_LAND_FLITS` with a default of 16 flits. The counter is spent per flit sent (header included) and is restored by RAPIDS `credit`/`add` returns.

Rationale: The default 16 matches the Hive-set `TILE_LAND_FLITS` default in TILE_SPEC §11.9. The counter is private to this cell and never goes negative; a packet is presented only while the counter covers the entire packet, header included.

9.1.3 Header Build

The packetizer builds write-format packets (`ptype=mem`, `op=0`) with the `TILE_SPEC` §11.5 tile header as the first payload beat(s). The header fields are:

Table 4.2: Result-packet header fields

Field	Value
Destination coordinate	Gatling-programmed drain destination (see below)
Source coordinate	This cell's external ID `{ext=1, level=0, X, Y, Z}`
`ptype`	`mem` (2)
`op`	0 (write)
Length	Payload bytes that follow
Target address	Programmed by the drain command (memory address in the sink channel)

Field	Value
Tag	Programmed sink channel

MAS decision on transport: The packetizer uses the TILE_SPEC §11.6 two-header model with the beat-chained header rule (owner ruling 2026-10-18 — every link keeps its own width):

- The delta route header is placed in TUSER of the head beat.
- The 128-bit tile header is placed in the first `ceil(128/DW)` payload beats, LSB-first — on the 64-bit local port: beat 0 = `header[63:0]`, beat 1 = `header[127:64]`.
- Payload beats follow.

The local port does not widen. This supersedes slice-1's provisional 64-bit TUSER packing and the 0.10 draft's 128-bit local-port widening.

9.1.4 Destination and Tag Programming

The cell does not choose the result destination. Gatling (the tetrad engine) commands the drain and tells the cell where to send it (TILE_SPEC §11.14; PRD §6.3).

Precedence: The operational drain destination is held in FLUSH_DEST (offset 0x54) and the sink-channel tag in FLUSH_TAG (offset 0x58). These are programmed by gatling (via `cfg` write or an extended CTRL flush command) and are what the packetizer uses when `CTRL.flush-and-send` is written. The legacy DEST register (offset 0x38) is **reserved** for the deferred cell→cell result path; it is not used for rapids-bound drains. DEST becomes operational only after the tile-spec cell→cell credit/grant extension lands (TILE_SPEC §10); until then, result traffic is RAPIDS-bound and FLUSH_DEST/FLUSH_TAG take precedence.

Rationale: This gives gatling a concrete mechanism to command a flush without requiring a new packet type or a sideband interface, while keeping the DEST register available for the future peer-cell path.

9.1.5 Strap Accounting

Each packet costs one header flit plus one flit per payload line today. The packetizer checks:

```
strap_counter >= 1 + payload_flits
```

before presenting `tvalid`. It decrements the counter by one per flit sent. RAPIDS returns credits as it drains the cell's landing space; the returns are `credit`/`add` packets that increment the strap counter.

9.2 Execution FSM

9.2.1 Pop → Issue → Push Loop

`cell_exec_fsm` runs the compute loop:

1. **Pop.** When enabled and not busy, pop `OP_LINES` input lines from `cell_mem_in`, advancing `IP` by one per line. Capture the source coordinate of the first line (all lines of one invocation are expected to come from the same source; if they differ, the first line's source is used for credit return).
2. **Issue.** Present the operand lines to the ALU and assert the issue `valid`. Wait for `ready`; hold operands and `func_sel` stable while `ready` is low.
3. **Push.** When the ALU asserts `result_valid` and the FSM asserts `result_ready`, push the result line into `cell_mem_out`, advancing `OP` by one.

The loop is pipelined: while one invocation computes, the FSM can pop and issue the next, as long as the ALU's elastic pipeline accepts it (II=1 while draining).

9.2.2 MAS Decision: Operand-Source Mux

MAS decision: The operand-source mux is configured by `OP_SRC`:

Table 4.3: `OP_SRC` encoding

`OP_SRC` value	`op_line_a` source	`op_line_b` source
0	input-memory pop	input-memory pop
1	`A`	`A`
2	`B`	`B`
3	`C`	`C`
4	`D`	`D`

The two sides are controlled by independent 3-bit fields in `OP_SRC`: `sel_a[2:0]` and `sel_b[2:0]`. Reset value is 0 (both operands from memory). The `T` register supplies `threshold_imm` for opcode 6 (compare/threshold).

9.2.3 IP/OP Pointer Behavior

`IP` and `OP` are 32-bit registers that auto-increment on pop and push respectively. They wrap at `IN_DEPTH` and `OUT_DEPTH`:

```
IP_next = (IP + 1) mod IN_DEPTH
OP_next = (OP + 1) mod OUT_DEPTH
```

CTRL.soft_reset clears both pointers to 0 and clears IRQ_STATUS. Software may also seed them through AXIL or cfg writes.

9.2.4 Gatling as the Cell ALU Sequencer (owner ruling 2026-10-19)

MAS decision: The slice-3 cell_exec_fsm remains the standalone bring-up driver for the cell, preserving the same POP → issue → PUSH client contract. In the tetrad-integrated tile, **gatling** (the tetrad barrel-core engine) sequences the cell's alu-infer ALU directly:

- POP reads operand line(s) from the cell's input memory and presents them to the ALU issue handshake.
- PUSH retires result line(s) from the ALU and writes them into the cell's result memory.
- FLUSH programs FLUSH_DEST/FLUSH_TAG and triggers the packetizer drain.
- CFG/ECTX/KICK/WAIT carry tetrad-level context and control.

Gatling never originates config traffic that belongs in the cell's cfg stream; cell programming remains in-band per the standing owner ruling 2026-10-17. The gatling PRD v0.2 owns the Xcell instruction encodings (CFG, POP, PUSH, ECTX, FLUSH, KICK, WAIT).

9.2.5 ALU Hookup

The FSM drives the ALU's elastic valid/ready interface per the alu-infer MAS:

Table 4.4: ALU🔄FSM seam signals

Signal	Direction	Purpose
alu_valid	FSM → ALU	Issue valid
alu_ready	ALU → FSM	Issue ready
op_line_a	FSM → ALU	Operand A line
op_line_b	FSM → ALU	Operand B line
threshold_imm	FSM → ALU	bf16 threshold immediate (from T)
func_sel	FSM → ALU	{opcode, sub-op, reserved}
acc_ctrl	FSM → ALU	CLEAR / ACCUMULATE / DRAIN for MAC
result_line	ALU → FSM	Result line
result_valid	ALU → FSM	Result valid

Signal	Direction	Purpose
result_ready	FSM → ALU	Result ready

The function/opcode table is defined in alu-infer PRD §3 and implemented per the alu-infer MAS. The cell MAS does not restate it.

9.2.6 Issue Tag and Same-Tag Alignment (ratified spec — RTL pending)

MAS decision: When the scheduler loads a new function into the ALU pipeline, an **8-bit tag** is assigned, common to every op of that invocation group that must complete together. A **tag FIFO** tracks the in-flight groups. Each op's result_valid asserts only when *every* result_valid of ops carrying the same tag has asserted — so ops that were scheduled together finish together, and the elastic pipeline's per-op readiness is gated to the group boundary.

Rationale (owner ruling): if two ops are scheduled in the same clock they should finish at the same time, and the scheduler programs the valid/ready behavior it wants (including feedback cases) through this tag grouping rather than through pipeline-depth matching. This takes care of the elastic alignment naturally: the group is atomic at the result boundary even though individual ops may have different internal latencies.

Status: ratified with the alu-infer work at rev 0.50; **not yet in the alu-infer RTL or the alu-infer MAS** — the tag assign/FIFO/align gate of Figure 2.3 is the specified target, not the current implementation.

9.2.7 MAC Accumulation

MAC behavior is inherited from the alu-infer contract: the FMA computes the bf16 product exactly in fp32, accumulates per-lane in fp32, and rounds to bf16 once per result-line presentation. The FSM sequences acc_ctrl (CLEAR at layer boundaries, ACCUMULATE for steps, DRAIN for the final values) without changing the ALU interface.

9.2.8 Soft Reset

CTRL.soft_reset (bit 1) pulses o_soft_rst for one cycle. The FSM responds by clearing IP and OP and dropping any in-flight issue request. The ALU's reset behavior is defined in the alu-infer MAS.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 Verification Plan

10.1 House Test Grid

The cell testbench follows the house gate / func / full grid:

Table 5.1: House test grid

Level	Scope	Typical seeds	Runtime target
gate	Smoke: elaboration, reset, register reset values, basic fill	1 seed	< 1 min
func	Directed feature tests across seeds	8 seeds	~5 min
full	Longer randomized runs, corner cases, coverage closure	32+ seeds	~30 min

10.2 Existing Tests

`cell/dv/tests/test_cell.py` already covers slice-1 behavior:

1. Elaboration + reset + register reset values over AXIL
2. AXIL and cfg-stream register access
3. Input-memory fill from a `ptype=mem` packet
4. Bad-packet drop behavior

The delta node inside `cell_top` is treated as a black box; slice-1 simulation drives the local rx port through the `test_mode` injection hook (`dut.test_rx_*`) so the testbench can supply cell traffic without fighting the node's continuous driver.

10.3 Planned Slice-2 Credit-Loop Test

The slice-2 test verifies the credit return path independently of the datapath:

1. Drive `mem` packets into the cell from multiple recorded sources (using the test-mode injection hook or a BFM on the local port).
2. Force the execution FSM to consume lines without running the ALU (or run a passthrough opcode).

3. Check that `cell_credit` emits `credit/add` packets whose destination equals the recorded source coordinate and whose length matches the configured batching (`CREDIT_BATCH_N` or residual).
4. Verify the 64-cycle partial-batch timeout by sending a partial batch and waiting for the automatic drain.
5. Verify the boot `credit/set` advertisement to `level=3` carrying `IN_DEPTH` flits.

The test forces straps so that result emission does not gate credit returns.

10.4 Planned Slice-3 ALU-Run Test

The slice-3 test verifies the pop→issue→push loop end to end:

1. Program `FUNC_SEL`, `OP_SRC`, `OP_LINES`, and persistent operands `A`–`D`, `T` via `cfg` or AXIL.
2. Fill the input memory with known operand lines.
3. Enable the FSM and run until `STATUS.busy` clears or an IRQ fires.
4. Read the result memory (via the packetizer drain or AXIL window).
5. Check results against the alu-infer golden model per opcode.

Coverage: opcode (8) × special-value classes × lane position × accumulator clear/hold/drain corners.

10.5 Verification Hooks

Table 5.2: Verification hooks

Hook	Use
`test_rx_*` / `test_mode`	Inject cell traffic without instantiating the delta mesh
`inc_lines_in/out`, `inc_credits`, `inc_pkts`	Count events independently of datapath
`o_todo_credit_drop_cnt` / `o_todo_cfg_read_drop_cnt`	Confirm slice-1 stubs are replaced in slice 2
`STATUS` / `IRQ_STATUS`	Observe empty/full, error, and strap-starved states

10.6 Assertions and Checks

- Credit return must match lines consumed, per source, within the batch timeout.
- Strap counter must never go negative; packetizer `tvalid` implies sufficient strap for the whole packet.
- ALU results retire in issue order.
- `IP` and `OP` wrap correctly at their respective depths.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 Register Map and MAS Decisions

11.1 Register Map

The v1 register map is identical to PRD §7, with two MAS-added operational registers for the gatling-programmed drain command.

Table 6.1: Cell register map

Offset	Register	Notes
0x00	CTRL	enable, soft reset, flush-and-send
0x04	STATUS	busy, input-empty, result-nonempty, strap-nonzero (RTL alias status_grant_nonzero)
0x08	FUNC_SEL	opcode (5b) + sub-op (2b) + reserved (2b)
0x0C	OP_LINES	input lines per function invocation
0x10	EMIT_BATCH_X	flits per result packet
0x14	CREDIT_BATCH_N	consumed lines per credit packet per source
0x18	IRQ_EN	done, error, strap-starved (RTL alias IRQ_GRANT_STARVED_BIT), credit-timeout enables
0x1C	IRQ_STATUS	write-1-clear status
0x20	A	persistent operand A
0x24	B	persistent operand B
0x28	C	persistent operand C
0x2C	D	persistent operand D
0x30	T	bf16 threshold immediate
0x34	OP_SRC	operand-source mux: sel_a[2:0], sel_b[2:0]

Offset	Register	Notes
0x38	DEST	deferred cell-destined destination
0x3C	IP	input-memory read pointer
0x40	OP	result-memory write pointer
0x44	CNT_LINES_IN	lines received
0x48	CNT_LINES_OUT	results produced
0x4C	CNT_CREDITS	credit packets returned
0x50	CNT_PKTS	result packets sent
0x54	FLUSH_DEST	**MAS-added:** gatling-programmed drain destination coordinate
0x58	FLUSH_TAG	**MAS-added:** gatling-programmed sink-channel tag

11.2 Bit Fields

11.2.1 CTRL (0x00)

Table 6.2: CTRL bit fields

Bit	Name	Purpose
0	EN	Enable execution FSM
1	SOFT_RST	Soft reset: clears IP, OP, IRQ_STATUS; pulse lasts one cycle
2	FLUSH	Packetizer flush-and-send: emit whatever is buffered now

11.2.2 STATUS (0x04) — read-only

Table 6.3: STATUS bit fields

Bit	Name	Purpose
0	BUSY	FSM busy (slice 3)
1	INPUT_EMPTY	Input memory empty
2	RESULT_NONEMPTY	Result memory not empty
3	STRAP_NONZERO	Strap counter > 0. Slice-1 RTL still names this bit STATUS_GRANT_NONZERO_BIT / status_grant_nonzero; the doc term is canonical.

11.2.3 FUNC_SEL (0x08)

Table 6.4: FUNC_SEL fields

Bits	Field	Purpose
4:0	opcode	ALU opcode (alu-infer PRD §3)
6:5	sub_op	Sub-opcode, e.g. min vs max, compare vs threshold
8:7	reserved	Reserved under the bf16-only model

11.2.4 OP_SRC (0x34)

Table 6.5: OP_SRC fields

Bits	Field	Encoding
2:0	sel_a	0=memory pop, 1=A, 2=B, 3=C, 4=D
5:3	sel_b	0=memory pop, 1=A, 2=B, 3=C, 4=D
31:6	reserved	0

11.2.5 IRQ_STATUS / IRQ_EN

Table 6.6: IRQ fields

Bit	Name	Purpose
0	DONE	Computation/drain complete
1	ERROR	Dropped packet or fatal protocol error
2	STRAP_STARVED	Packetizer has data but no strap. Slice-1 RTL still names this bit IRQ_GRANT_STARVED_BIT; the doc term is canonical.
3	CREDIT_TIMEOUT	Partial-batch credit timer fired (informational)

11.3 MAS Decisions Revisited

Table 6.7: MAS decisions and their hardware homes

Decision	Where in this book	Register / parameter
Input/result memory depth = 32 lines	Chapter 4.2	IN_DEPTH, OUT_DEPTH parameters
Boot advertisement = 32 flits	Chapter 4.2	Derived from IN_DEPTH
Credit batch default = 1	Chapter 4.2	CREDIT_BATCH_N register
Partial-batch timeout = 64 cycles	Chapter 4.2	Hard-coded in cell_credit
Result batch flush conditions	Chapter 4.3	EMIT_BATCH_X, CTRL.FLUSH
Strap default = 16 flits	Chapter 4.3	TILE_LAND_FLITS parameter
Local-port width stays at delta DATA_WIDTH (64 bits today)	Chapter 3, Chapter 4.3	DATA_WIDTH parameter
Gatling-programmed	Chapter 4.3	FLUSH_DEST, FLUSH_TAG

Decision	Where in this book	Register / parameter
drain		registers

11.4 RTL Naming Divergence (slice 1)

Slice-1 RTL (`cell_pkg.sv`, `cell_cfg_regs.sv`) predates the 2026-10-18 ruling that renamed result-path grants to straps. The canonical MAS/PRD terms are **strap counter**, `STRAP_NONZERO`, and `STRAP_STARVED`; the slice-1 RTL identifiers remain:

- `STATUS_GRANT_NONZERO_BIT` / `status_grant_nonzero` for `STATUS` bit 3
- `IRQ_GRANT_STARVED_BIT` for `IRQ_STATUS`/`IRQ_EN` bit 2

A future RTL pass (slice 2/3) should rename these to the strap terms. Until then, the register map above states the canonical names and notes the RTL aliases.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

12 References

12.1 Normative Specifications

- **cell PRD** — `projects/components/compute-eng-ip/cell/docs/PRD.md`.
  The governing product definition. The MAS implements PRD §12 open questions and cites §§3, 6, 7 throughout.

- **TILE_SPEC §11** — `projects/components/compute-eng-ip/tile/docs/TILE_SPEC.md`.
  Defines the packet header (§11.5), transport binding (§11.6), payload types (§11.2), destination coordinate (§11.3), end-to-end credits and straps (§§11.9–11.10), and result flush (§11.14).

- **alu-infer PRD §3** — `projects/components/compute-eng-ip/alu-infer/PRD.md`.
  Normative ALU function set, element model (bf16), and MAC accumulation semantics.

- **alu-infer MAS** — `projects/components/compute-eng-ip/alu-infer/docs/alu_infer_mas/alu_infer_mas_index.md`.
  Implementation-intent book for the ALU block: module tree, function rows, accumulator, ALU↔️FSM interface, and cycle behavior.

12.2 RTL Sources

- **cell_top.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_top.sv`.
  Slice-1 top-level: delta node, rx demux, registers, memories.

- **cell_rx.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_rx.sv`.
  Slice-1 header parse/demux with provisional 64-bit TUSER packing.

- **cell_cfg_regs.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_cfg_regs.sv`.
  Slice-1 AXIL + cfg-stream register file.

- **cell_mem_in.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_mem_in.sv`.
  Slice-1 input memory with per-line source tag.

- **cell_mem_out.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_mem_out.sv`.
  Slice-1 result memory.

- **cell_pkg.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_pkg.sv`.
  Shared types, coordinate layout, register offsets, and slice-1 compact header offsets.

- **delta_mesh_cell.sv** — `projects/components/fabric-gen-ip/delta/rtl/delta_mesh_cell.sv`.
  The node the cell instantiates; defines the 9-port unpacked-array interface and `P_LOCAL = 8`.

12.3 Verification

- **test_cell.py** — `projects/components/compute-eng-ip/cell/dv/tests/test_cell.py`.
  Existing slice-1 cocotb test suite; the base for the slice-2/3 credit-loop and ALU-run tests.

12.4 House Style and Build

- **Block diagrams handbook** — `vault/handbook/authoring/block-diagrams.md`.
  Binding house style for the whole-cell and credit-loop figures.

- **alu-infer MAS PDF generator** — `projects/components/compute-eng-ip/alu-infer/docs/generate_mas_pdf.sh`.
  The build script this MAS generator is modeled on.

Last Updated: 2026-10-18